

K L (Deemed to be) UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY

BACHELOR OF TECHNOLOGY

In

“ONLINE FOOD DELIVERY PLATFORM”

*Design and Development of a Full-Stack Food Ordering, Delivery Tracking and
Group Split-Bill Platform*

A Project Report Submitted in partial fulfillment of the requirements for the award of
the degree of Bachelor of Technology

By

MOSANGI VIJAYA LAKSHMI	2520090201
POTLABATHINI SHASHANTH	2520090211
D PAWAN KUMAR	2520090102

Under the Esteemed Guidance of

Dr. M. Gayatri

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University)

KLH Bachupally Campus | Hyderabad, Telangana

Academic Year 2026–2027 | Section 13 | Team 15

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY



Declaration

The project report entitled “Online Food Delivery Platform” is a record of the bonafide work carried out by MOSANGI VIJAYA LAKSHMI – 2520090201, POTLABATHINI SHASHANTH – 2520090211, and D PAWAN KUMAR – 2520090102, of Section 13, Team 15, submitted in partial fulfillment of the requirements for the award of the Bachelor of Technology degree. The project presents the analysis, design, implementation, database development, testing, and documentation of a full-stack web platform that connects customers, restaurants, delivery partners, and an administrative role. The work incorporates React.js, Node.js, Express.js, MySQL, REST APIs, Socket.IO, and the proposed Google Maps and Razorpay integrations, together with the Group Ordering with Live Split Bill feature. The matter contained in this report has been prepared for the academic project and has not been knowingly copied from another department, university, or institute.

MOSANGI VIJAYA LAKSHMI – 2520090201

POTLABATHINI SHASHANTH – 2520090211

D PAWAN KUMAR – 2520090102

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled “Online Food Delivery Platform” is a bonafide work done and submitted by MOSANGI VIJAYA LAKSHMI – 2520090201, POTLABATHINI SHASHANTH – 2520090211 and D PAWAN KUMAR – 2520090102, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2026–2027, under the guidance of Dr. M. Gayatri.

Signature of the Guide

Signature of the Course Coordinator	Signature of the HOD	Department Seal
--	----------------------	-----------------

ACKNOWLEDGEMENT

The successful completion of the Online Food Delivery Platform project was made possible through the guidance, encouragement, and support received during the planning, design, implementation, testing, and documentation stages. We express our sincere gratitude to our project guide, Dr. M. Gayatri, Department of Computer Science and Engineering, for providing continuous technical guidance, constructive suggestions, and motivation throughout the project. Her support helped us refine the system requirements, database model, role-based workflows, real-time communication design, and the Group Ordering with Live Split Bill feature.

We are grateful to the faculty members of the Department of Computer Science Engineering and the Department of Computer Science and Information Technology for providing the academic environment, laboratory facilities, and learning resources required to complete this work. We also acknowledge the value of developer documentation and learning resources associated with React.js, Node.js, Express.js, MySQL, Socket.IO, Google Maps Platform, Razorpay, Postman, and related development tools that informed the technical implementation.

We thank our teammates, classmates, friends, and family members for their cooperation during requirement analysis, database design, coding, debugging, UI refinement, testing, and report preparation. Their feedback helped us identify usability issues and improve the clarity of the customer, restaurant, delivery-partner, and administrative workflows. Finally, we thank everyone who reviewed the application during demonstrations and provided suggestions for future enhancements.

MOSANGI VIJAYA LAKSHMI – 2520090201

POTLABATHINI SHASHANTH – 2520090211

D PAWAN KUMAR – 2520090102

ABSTRACT

The Online Food Delivery Platform, named FoodFlow for the application interface, is a full-stack web system designed to digitize the complete food ordering and delivery workflow in one integrated platform. The system connects customers, restaurants, delivery partners, and an administrative role through role-based web interfaces. Customers can discover restaurants, view menus, add items to a cart, place orders, choose a delivery address, initiate payment, and monitor order status. Restaurant users can maintain menus and process incoming orders. Delivery partners can view eligible deliveries, accept assignments, update pickup and delivery states, and share location information when tracking is enabled. The administrative module provides platform-level monitoring for restaurant onboarding, commission information, and operational order data.

The technical stack combines React.js and Vite for the user interface, Node.js and Express.js for backend services and REST APIs, and MySQL 8.0 for persistent relational data. The platform uses Socket.IO for real-time events such as order-status changes, shared group-cart updates, payment-status updates, and delivery-location events. Google Maps API is planned for map-based location visualization, while Razorpay is the intended online payment service for a gateway-backed deployment. The academic prototype also supports a demonstration payment workflow so that the state transitions can be verified without performing real-money transactions during evaluation.

A major differentiating feature is Group Ordering with Live Split Bill. Multiple participants can create or join a group order, select the restaurant, contribute food items to a shared cart, and view member-level contributions. The backend records each cart contribution with the participating user identifier and calculates individual subtotals from quantity and unit price. A group-payment record is maintained for each participant. The group becomes ready for placement only after all required payment records reach the paid state. This feature demonstrates collaborative real-time state management, relational integrity, transaction-oriented design, and integration between frontend events, backend APIs, and database persistence.

Project identity: Batch/Team 15 | Section 13 | Academic Year 2026–27 | Guide: Dr. M. Gayatri | Core stack: React, Node.js, Express, MySQL, Socket.IO

INDEX

S. No.	Chapters	Topics	Page.no
—	Acknowledgement	Project acknowledgements and institutional support	4
—	Abstract	Project abstract and differentiating feature	5
1	Introduction	1.1 Background; 1.2 Problem Statement; 1.3 Scope; 1.4 Objectives; 1.5 Importance; 1.6 Target Users; 1.7 Existing vs Proposed	8–18
2	System Requirements	2.1 Hardware; 2.2 Software; 2.3 Development Tools; 2.4 Functional Requirements; 2.5 Non-Functional Requirements; 2.6 Feasibility	19–24
3	Technology Stack	3.1 Front-End; 3.2 React/Vite; 3.3 Back-End; 3.4 REST APIs; 3.5 Database; 3.6 Authentication; 3.7 Real-Time; 3.8 Payment; 3.9 Version Control; 3.10 External Services	25–37
4	System Architecture	4.1 High-Level; 4.2 Layers; 4.3 Request Journey; 4.4 Role Architecture; 4.5 Customer; 4.6	38–48

S. No.	Chapters	Topics	Page.no
		Restaurant; 4.7 Delivery; 4.8 Admin; 4.9 Real-Time; 4.10 Deployment; 4.11 Security	
5	Design	5.1 System Design; 5.2 Module Decomposition; 5.3 Schema; 5.4 Entities; 5.5 Relationships; 5.6 ER Diagram; 5.7 Group Split Bill; 5.8 Order Status; 5.9 UI Navigation; 5.10 API Contracts	49–58

Reading note: The page ranges below correspond to the planned printed report sequence. Detailed subsection pages, figures, tables, testing evidence, and appendices are included in the chapters that follow.

INDEX – CONTINUED

S. No.	Chapters	Topics	Page.no
6	Implementation	6.1 Structure; 6.2 Setup; 6.3 Customer; 6.4 Restaurant; 6.5 Delivery; 6.6 Admin; 6.7 Group Order; 6.8 Split Bill; 6.9 API Service; 6.10 Socket.IO; 6.11 Validation; 6.12 Error Handling	59–72
7	Features	7.1 Main Features; 7.2 Authentication; 7.3 Restaurant Discovery; 7.4 Cart/Checkout; 7.5 Tracking; 7.6 Restaurant Ops; 7.7 Delivery; 7.8 Group Ordering; 7.9 Split Bill/Admin	73–81
8	Testing	8.1 Strategy; 8.2 API/Postman; 8.3 Integration; 8.4 Validation; 8.5 Test Environment; 8.6 Test Cases; 8.7 Results	82–92
9	Deployment	9.1 Deployment Steps; 9.2 Environment Configuration; 9.3 Local Run; 9.4 Front-End Hosting; 9.5 Back-End Hosting; 9.6 Production Checklist	93–98
10	Challenges & Limitations	10.1 Challenges Faced; 10.2 Solutions Applied; 10.3 Current	99–103

S. No.	Chapters	Topics	Page.no
		Limitations and Known Issues	
11	Future Enhancements	11.1 Planned Features; 11.2 Payment/Map Integrations; 11.3 Analytics; 11.4 Performance/Security; 11.5 Intelligent Enhancements	104–108
12	Conclusion	12.1 Summary; 12.2 Achievements; 12.3 Skills Learned	109–111
13	References	Books, tutorials, APIs, official documentation, developer resources	112–114
14	Appendices	A. Screenshots; B. Code Snippets; C. Installation; D. User Guide; E. Academic Review Checklist; F. Feedback/Signatures	115–124

Reading note: The page ranges below correspond to the planned printed report sequence. Detailed subsection pages, figures, tables, testing evidence, and appendices are included in the chapters that follow.

CHAPTER 1 – INTRODUCTION

The chapter 1 – introduction discussion focuses on the project turns a fragmented food-ordering workflow into a single digital ecosystem. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, customers can discover restaurants, select menu items, place orders, and follow delivery status while partner roles process the same order record. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, a database-backed platform is a suitable case study for primary keys, foreign keys, joins, CRUD operations, state transitions, and real-time communication. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

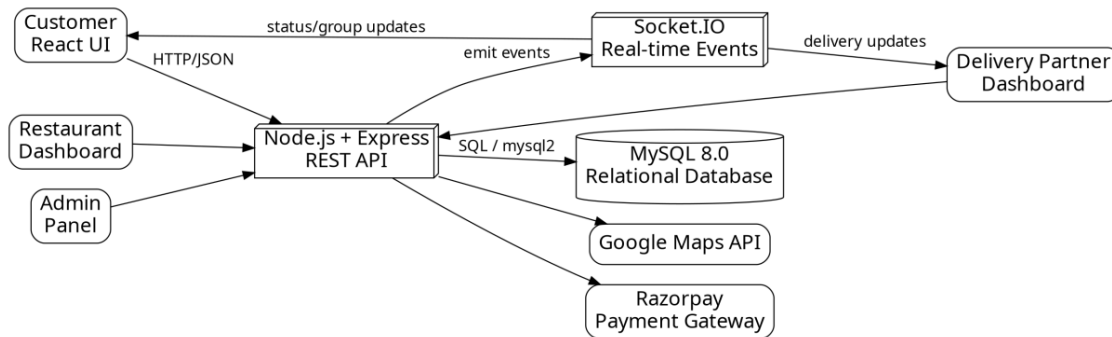


Figure 1.1 High-level architecture of the FoodFlow platform

1.1 Background of the Project

1.1 Background of the Project is considered an important part of the FoodFlow project because online food ordering requires coordinated information between customers, restaurants, delivery partners, and administrators. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow stores restaurants, menus, carts, orders, addresses, delivery assignments, tracking records, and group-order data in one relational system. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the

complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the project combines DBMS concepts with web development and event-driven backend communication. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Centralized information reduces duplicate manual entry.
- Role-based screens separate operational responsibilities.
- Real-time events improve the visibility of changing order states.

1.1 Background – Digital Food Service Workflow

The 1.1 background – digital food service workflow subsection explains a digital order is a sequence of linked records rather than a single form submission. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the platform preserves relationships between a customer, restaurant, menu items, order items, address, delivery assignment, and payment state. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the report models these relationships explicitly so that data remains traceable throughout the order lifecycle. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Customer actions create structured records.
- Restaurant actions move the order through preparation states.
- Delivery actions complete the final states.

1.2 Problem Statement

The 1.2 problem statement discussion focuses on phone-based or fragmented ordering can lack shared order visibility, centralized payment, and organized delivery coordination. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow provides one workflow in which the same order identifier is used by customer, restaurant, and delivery views. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the problem is therefore treated as an integration problem involving data consistency, role authorization, and timely state propagation. The approach provides a direct mapping between the stated project objectives and

demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Reduce disconnected communication.
- Maintain a common order record.
- Provide status and payment visibility.

1.2 Problem Statement – Group Ordering Gap

1.2 Problem Statement – Group Ordering Gap is considered an important part of the FoodFlow project because group orders are difficult when each participant tracks items and payments separately. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow introduces a shared cart in which each added item is linked to the contributing participant and later mapped to a split payment record. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the database can then derive participant subtotals and the frontend can show which members are paid or pending. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Shared cart contributions remain attributed to users.
- The total is the sum of participant subtotals.
- Group placement depends on the payment state.

1.3 Scope of the Project

The 1.3 scope of the project subsection explains the system scope covers customer ordering, restaurant menu and order processing, delivery fulfilment, administration, and group ordering. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the web application is designed to operate as a local academic prototype while leaving production credentials and cloud deployment as configurable extensions. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is scope boundaries keep the semester project demonstrable while preserving a clear path for future expansion. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Web interface for four roles.
- MySQL-backed persistence.
- Socket.IO-ready real-time workflows.

1.3 Scope – Data and Operational Boundaries

The 1.3 scope – data and operational boundaries discussion focuses on the core database scope includes users, restaurants, menu items, orders, order items, addresses, delivery records, group orders, members, cart items, and payments. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow focuses on food delivery operations rather than unrelated travel, grocery, or marketplace domains. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, data collected by the platform is limited to what is needed to support the demonstrated workflow and future integration points. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Relational tables support referential integrity.
- Order status is controlled by operational roles.
- External integrations remain separately configurable.

1.4 Objectives

1.4 Objectives is considered an important part of the FoodFlow project because the primary objective is to build an integrated and role-based food delivery system. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the project implements restaurant discovery, menu browsing, cart and checkout, restaurant processing, delivery fulfilment, administration, live updates, and group split-bill logic. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, objectives are mapped to demonstrable UI screens, database structures, backend endpoints, and testing evidence. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Provide clear role-based workflows.
- Use MySQL for persistent relational data.
- Demonstrate real-time and split-payment scenarios.

1.5 Importance of the Application

The 1.5 importance of the application subsection explains the project demonstrates how database design can support a real transaction-oriented web application. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the same data model can answer operational questions such as order status, menu availability, delivery assignment, participant contribution, and payment completion. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the academic value comes from integrating DBMS, backend engineering, frontend design, and distributed event communication in one project. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Primary/foreign keys and joins.
- CRUD and validation.
- State and event synchronization.

1.6 Target Users / Audience

The 1.6 target users / audience discussion focuses on different users require different information and permissions. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, customer, restaurant, delivery, and admin workflows are separated so that each role sees the actions relevant to its responsibility. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, role-specific interfaces also provide a clean demonstration path during evaluation because a reviewer can observe one lifecycle from multiple viewpoints. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Customer: discovery, cart, checkout, tracking.
- Restaurant: menu and order operations.
- Delivery: assignment, pickup, delivery, location.
- Admin: onboarding, commission, monitoring.

1.7 Existing System vs Proposed System

1.7 Existing System vs Proposed System is considered an important part of the FoodFlow project because traditional phone or fragmented workflows provide limited automation and weak shared visibility. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow consolidates ordering, preparation, delivery, and group payment information in a centralized platform with a shared database. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the proposed design is not described as a replacement for commercial products; it is an academic implementation that demonstrates comparable workflow concepts with a distinctive group split-bill focus. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Aspect	Fragmented Workflow	FoodFlow Project
Ordering	Phone/manual or separate screens	Web ordering with cart and checkout
Status	Manual communication	Controlled order lifecycle + Socket.IO events
Delivery	Separate coordination	Delivery-partner workflow
Group payment	Manual calculation	Shared cart + member-level payment state
Data	Scattered records	Central MySQL model

CHAPTER 2 – SYSTEM REQUIREMENTS

The chapter 2 – system requirements subsection explains the project needs enough computing resources to run the frontend, backend, database, and browser tools together. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the recommended student setup is a modern Windows laptop or desktop with adequate RAM and SSD storage. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is resource planning is important because local development may run several processes at the same time. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

2.1 Hardware Requirements

The 2.1 hardware requirements discussion focuses on hardware requirements are modest because the application is a web prototype rather than a compute-intensive simulation. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the development machine should comfortably run VS Code, Node.js, MySQL, a browser, and API tools concurrently. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, performance during demonstration depends on keeping background services and browser tabs within the available memory. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

Component	Recommended Academic Setup	Purpose
CPU	Modern multi-core processor	Run Node, MySQL, browser and editor
RAM	8 GB or more	Concurrent development services
Storage	SSD with free space	Project files and dependencies
Network	Stable internet for integrations	Package installation / external APIs
Display	1366×768 or better	Responsive UI demonstration

2.2 Software Requirements

2.2 Software Requirements is considered an important part of the FoodFlow project because the software environment provides the runtime, database, browser, editor, and testing utilities required by the project. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow uses React.js and Vite on the frontend, Node.js and Express.js on the backend, MySQL for persistence, and Socket.IO for real-time communication. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the environment should keep frontend and backend configuration separate so that local URLs and credentials remain manageable. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Software	Role in FoodFlow
Windows 10/11	Current development operating system

Software	Role in FoodFlow
VS Code	Code editing and integrated terminal
Node.js + npm	Runtime and package management
MySQL 8.0	Relational database
Chrome / modern browser	UI demonstration and debugging
Postman	API testing and verification

2.3 Development Tools and Frameworks

The 2.3 development tools and frameworks subsection explains frameworks reduce repetitive infrastructure work and let the team focus on application-specific logic. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, Vite supports fast frontend development, React supports component-based UI, Express defines backend routes, mysql2 provides database access, and Socket.IO supports event communication. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is tool selection also affects team productivity because the project can be developed and tested using widely available academic tools. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- React.js + Vite for frontend.
- Node.js + Express.js for backend.
- MySQL + mysql2 for persistence.
- Socket.IO + socket.io-client for live events.
- Postman and browser developer tools for testing.

2.4 Functional Requirements

The 2.4 functional requirements discussion focuses on functional requirements describe what the platform must allow users to do. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow requires role-specific authentication, restaurant discovery, menu viewing, cart operations, order placement, preparation updates, delivery fulfilment, administrative monitoring, and group split-bill operations. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, each function should result in a predictable UI action, backend request, database update, and status response. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

Role	Core Functions
Customer	Login, browse, cart, address, order, pay, track
Restaurant	Login, menu CRUD, order queue, preparation states
Delivery	Login, availability, accept, pickup, deliver, location
Admin	Onboarding, commission view, platform order monitoring

2.5 Non-Functional Requirements and Feasibility

2.5 Non-Functional Requirements and Feasibility is considered an important part of the FoodFlow project because quality requirements address security, maintainability, responsiveness, reliability, usability, and future scalability. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the project is technically feasible because the main stack is available locally and the core data model is relational and well suited to MySQL. The implementation separates presentation, business processing, persistence, and real-time

communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, economic and operational feasibility are also supported by the use of open-source development tools and a demonstration payment workflow for academic evaluation. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Usability: consistent navigation and role-based dashboards.
- Maintainability: modular frontend/backend separation.
- Reliability: validation and controlled states.
- Security: hashed passwords and token-based sessions.
- Feasibility: local execution without mandatory real-money transactions.

The chapter 3 – technology stack subsection explains the selected stack separates UI development, backend services, persistence, and real-time communication. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the combination supports the project requirement for a modern full-stack web application with a relational database. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is each technology is documented in terms of its role in FoodFlow rather than as a generic technology survey. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

3.1 Front-End Design

The 3.1 front-end design discussion focuses on React.js was selected for component-based interface development and Vite was selected for the local development workflow. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow uses reusable page and component patterns for home, restaurants, menus, cart, orders, dashboards, and Group Order. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, a component-oriented frontend helps keep presentation logic separate from API calls and makes role-specific screens easier to maintain. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Functional pages and reusable UI components.

- React hooks for state and effects.
- Responsive card and dashboard layouts.

3.1 Front-End – Navigation and Visual Language

3.1 Front-End – Navigation and Visual Language is considered an important part of the FoodFlow project because a food delivery UI must make restaurant discovery, ordering, tracking, and group collaboration easy to understand. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow uses a warm off-white and orange visual theme, cards, badges, action buttons, and role-specific navigation labels. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, visual consistency reduces cognitive load and also makes screenshots more suitable for academic documentation. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

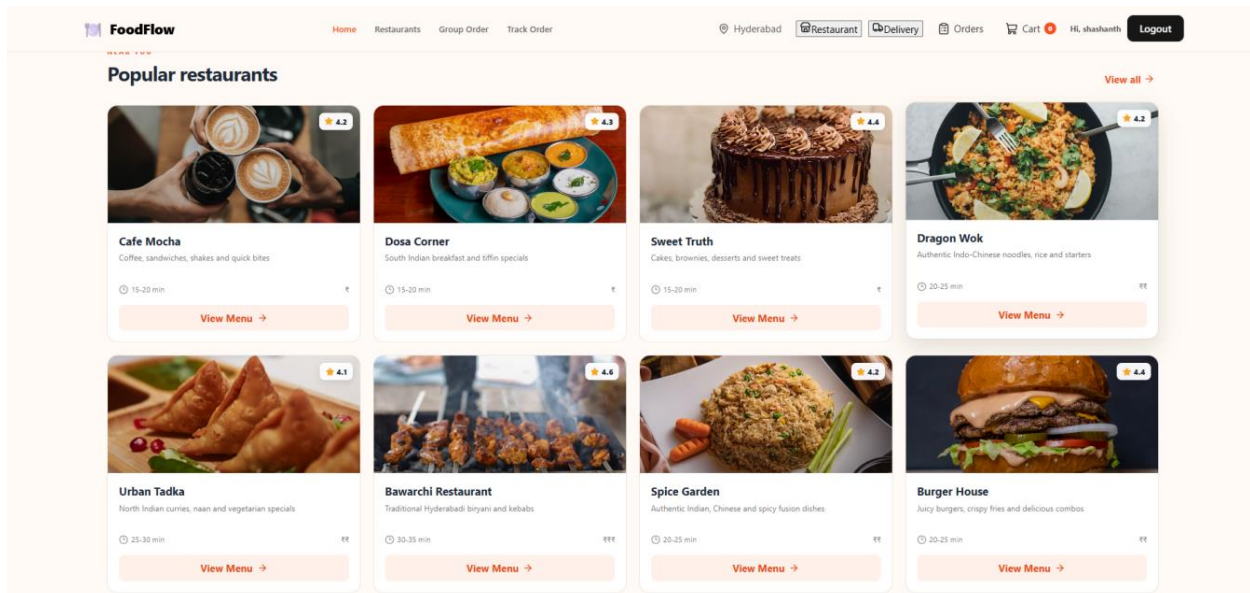


Figure 3.1 FoodFlow restaurant discovery interface screenshot

3.2 React State and Component Logic

The 3.2 react state and component logic subsection explains front-end state represents information currently visible or editable by a user. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow maintains state for authentication, cart contents, selected restaurant, order records, group members, shared cart items, and payment status. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is state changes are triggered by user actions, REST responses, or Socket.IO events, after which React re-renders the affected components. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- useState for local UI data.
- useEffect for initial loads and event subscriptions.
- useMemo for derived totals when useful.
- Service functions keep HTTP details out of visual components.

3.3 Back-End Design

The 3.3 back-end design discussion focuses on Node.js provides the runtime for backend JavaScript and Express.js defines a lightweight HTTP API layer. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow organizes routes and controller-style logic around authentication, restaurants, menus, orders, delivery, group orders, and payments. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the backend is the central coordination point because it validates requests, accesses MySQL, enforces role rules, and emits real-time events. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Express routes map methods to operations.
- Controllers perform validation and SQL work.
- Middleware can enforce authentication and roles.

3.4 REST API Design

3.4 REST API Design is considered an important part of the FoodFlow project because REST endpoints provide a stable contract between the React frontend and the backend. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow uses resource-oriented endpoints for restaurants, menu items, orders, delivery operations, group orders, and payment state. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, consistent request and response structures make it easier to test services independently with tools such as Postman. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and

transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Method	Representative Endpoint	Purpose
POST	/api/auth/login	Authenticate user
GET	/api/restaurants	List restaurants
GET	/api/restaurants/:id/menu	Get menu
POST	/api/orders	Create order
PATCH	/api/orders/:id/status	Update order status
POST	/api/group-orders	Create group order
POST	/api/group-orders/:id/items	Add shared-cart item

3.5 Database Design – MySQL 8.0

The 3.5 database design – mysql 8.0 subsection explains MySQL is appropriate because FoodFlow contains structured entities with clear relationships and transactional state changes. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the schema separates user, restaurant, menu, order, delivery, group, and payment data so that each record has a clear purpose. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is primary keys identify records and foreign keys maintain valid relationships between dependent rows. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

Entity	Purpose
users	Identity, role, password hash
restaurants	Restaurant profile and owner relationship
menu_items	Food catalogue and availability
orders / order_items	Normal order header and line items
delivery_assignments	Delivery responsibility and status
group_orders / group_members	Collaborative order membership
group_cart_items	Participant-level shared cart contributions
group_payments	Member payment state

3.5 Database Design – Normalization and Integrity

The 3.5 database design – normalization and integrity discussion focuses on normalization reduces redundant data and helps prevent update anomalies. This requirement influences both the user interface and the data model, because a visible

action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow keeps repeated concepts such as restaurant profile, menu item, and user identity in separate tables and relates them through keys. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, referential integrity ensures that an order cannot reference an invalid customer or restaurant and that group records remain connected to their participants. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- 1NF: atomic values and repeatable rows represented as child records.
- 2NF/3NF: attributes depend on the correct key.
- Foreign keys preserve valid relationships.
- Unique constraints can protect emails or group codes.

3.5 Database Design – Querying and Indexing

3.5 Database Design – Querying and Indexing is considered an important part of the FoodFlow project because the platform needs queries that retrieve dashboards, menus, order history, delivery queues, and group payment summaries. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, indexes on common lookup fields such as email, restaurant_id, order_id, group_order_id, and status can reduce lookup cost as the dataset grows. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, SQL JOINS combine normalized tables to generate the complete view required by the frontend. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Query Need	Tables Commonly Involved
Restaurant menu	restaurants + menu_items
Order details	orders + order_items + menu_items + addresses
Delivery queue	orders + delivery_assignments + restaurants
Group split bill	group_orders + group_members + group_cart_items + group_payments

3.6 Authentication and Authorization

The 3.6 authentication and authorization subsection explains users must be authenticated before protected operations such as order management or restaurant administration are allowed. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the project uses bcryptjs for password hashing and JWT-style signed tokens for session representation. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is authorization then applies role checks so a restaurant user cannot perform delivery-only or administrator-only actions. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Never store plaintext passwords.
- Validate tokens on protected API requests.
- Check role before sensitive operations.
- Keep client storage limited to required session state.

3.7 Real-Time Communication with Socket.IO

The 3.7 real-time communication with socket.io discussion focuses on some food delivery states are more useful when the interface updates immediately instead of waiting for a manual refresh. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow uses Socket.IO as an event channel for order-status updates, group membership changes, shared-cart updates, payment-state changes, and delivery-location events. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the backend emits an event after a meaningful state change and connected clients use the payload to refresh or update local state. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

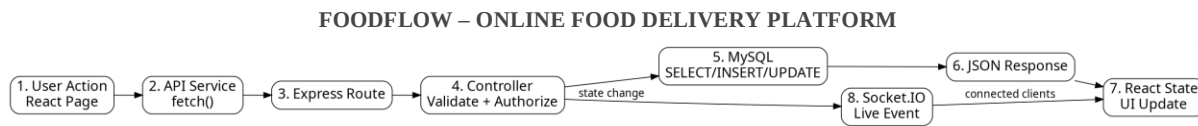


Figure 3.2 Request journey and real-time event path

3.8 Payment Model

3.8 Payment Model is considered an important part of the FoodFlow project because payment is represented as an application state rather than being embedded as an uncontrolled UI action. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the project uses a demonstration payment workflow for academic verification while retaining a clear integration point for Razorpay in a credentialed deployment. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, for group orders, each member gets a separate payment record so the system can show pending and paid contributions independently. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.



Figure 3.3 Group Ordering with Live Split Bill flow

3.9 Version Control and 3.10 External Services

The 3.9 version control and 3.10 external services subsection explains version control keeps source changes traceable and supports team collaboration. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow can use Git and GitHub for repository management, while Google Maps and Razorpay are treated as external services accessed through configured credentials. The current academic build is intended for local demonstration on Windows,

VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is sensitive files such as environment variables and API keys should be excluded from the repository and managed through local or hosting configuration. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

Service / Tool	Role
Git + GitHub	Version history and collaboration
Google Maps API	Map visualization and location tracking
Razorpay	Payment gateway integration point
Postman	API request testing
VS Code	Development environment

CHAPTER 4 – SYSTEM ARCHITECTURE

The chapter 4 – system architecture discussion focuses on the system follows a layered architecture so that frontend, backend, database, and external integrations have clear responsibilities. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow uses React/Vite, Node/Express, MySQL, Socket.IO, and configurable external service connectors. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, layered separation improves maintainability and makes the request lifecycle easy to explain during a viva or review. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

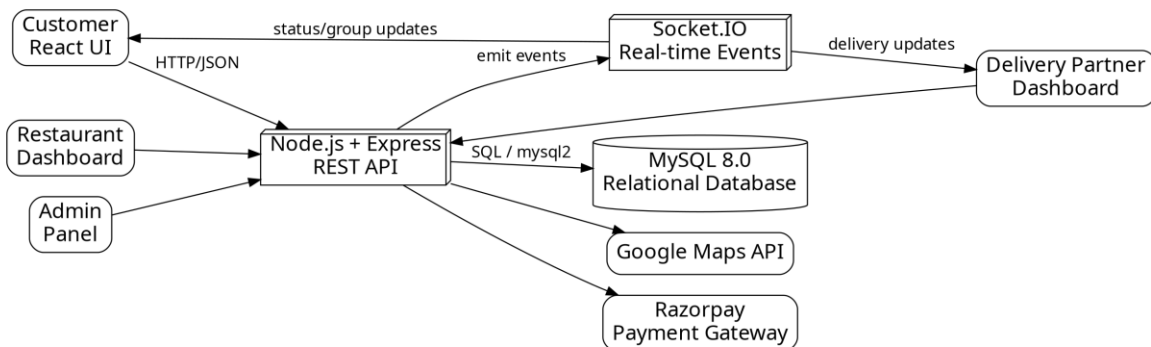


Figure 4.1 High-level FoodFlow architecture

4.2 Presentation Layer

4.2 Presentation Layer is considered an important part of the FoodFlow project because the presentation layer receives user actions and displays application state. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, React components render restaurant cards, menu lists, cart items, dashboards, status timelines, payment states, and group-order views. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete

platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, frontend components should not directly manage database operations; they call the API service layer instead. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Reusable components reduce UI duplication.
- Role-based routing controls page access.
- Loading and error states improve usability.

4.2 Service / Business Layer

The 4.2 service / business layer subsection explains the backend service layer interprets requests and applies business rules before writing persistent data. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, Express routes pass data to controllers and service functions that validate inputs, check permissions, update MySQL, and prepare responses. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is keeping business rules on the server prevents the browser from becoming the only authority for prices, roles, or state transitions. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Validate IDs and numeric quantities.
- Recalculate totals on the server.
- Reject invalid status transitions.

4.2 Data Layer

The 4.2 data layer discussion focuses on the data layer provides durable storage for all core business records. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, MySQL stores users, restaurants, menus, orders, delivery assignments, tracking history, group orders, members, shared cart items, and payments. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the frontend never writes directly to MySQL; all access passes through the backend API. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Primary keys identify records.
- Foreign keys preserve relationship integrity.
- SQL queries support dashboards and reports.

4.3 Customer Architecture

4.3 Customer Architecture is considered an important part of the FoodFlow project because the customer workflow is centered on discovery, ordering, payment, and tracking. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, customer pages call restaurant, menu, cart, order, address, and group-order APIs while Socket.IO provides immediate updates where needed. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the data path from customer click to database row can be traced through the API service, controller, SQL query, and JSON response. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

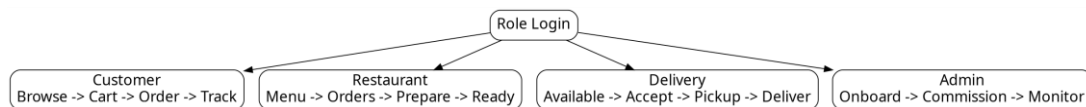


Figure 4.2 Role-based workflow overview

4.4 Restaurant Architecture

The 4.4 restaurant architecture subsection explains restaurant users need a queue-oriented interface rather than a shopping-oriented interface. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow restaurant screens focus on menu management and the controlled progression of incoming orders from confirmation to preparation and pickup readiness. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid

credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is restaurant ownership is checked before menu or order records are changed so one restaurant cannot modify another restaurant's data. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Manage item name, price, availability.
- Review incoming orders.
- Move preparation status forward.

4.5 Delivery Partner Architecture

The 4.5 delivery partner architecture discussion focuses on delivery partners need a concise operational view of available and assigned deliveries. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the delivery workflow exposes assignment, acceptance, pickup, out-for-delivery, and delivered states plus availability and optional location updates. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the backend uses assignment records so an order can be tied to a partner without duplicating customer or restaurant information. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Availability indicates whether a partner can receive work.
- Pickup is allowed only for eligible states.
- Location updates can be linked to active assignments.

4.6 Admin Architecture

4.6 Admin Architecture is considered an important part of the FoodFlow project because administrators need platform-level visibility rather than access to one customer or restaurant context. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the admin layer is designed for restaurant onboarding, commission tracking, and platform-wide order monitoring. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, these operations are intentionally separated from ordinary customer flows because they can affect multiple entities and require stronger authorization. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Review new restaurant records.
- Monitor order and commission information.
- Use role-protected admin routes.

4.7 Real-Time Event Architecture

The 4.7 real-time event architecture subsection explains real-time features require a separate event path in addition to request-response APIs. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, Socket.IO rooms or event identifiers can isolate a specific order or group so only interested clients receive updates. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is events are emitted after the authoritative database change, making MySQL the source of persistence and Socket.IO the source of

immediate notification. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

Event Family	Example Event	Meaning
Order	order:status	Order lifecycle changed
Delivery	delivery:location	Partner location updated
Group	group:item-added	Member added food
Group	group:cart-updated	Shared cart refreshed
Payment	group:payment-updated	Member payment changed

4.8 Deployment Architecture

The 4.8 deployment architecture discussion focuses on the academic prototype can run with separate frontend, backend, and database processes on one machine. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, a production-style deployment can split these components across a static frontend host, Node-compatible backend host, and managed MySQL service. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, environment variables are used to swap local addresses for deployment addresses without rewriting application logic. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

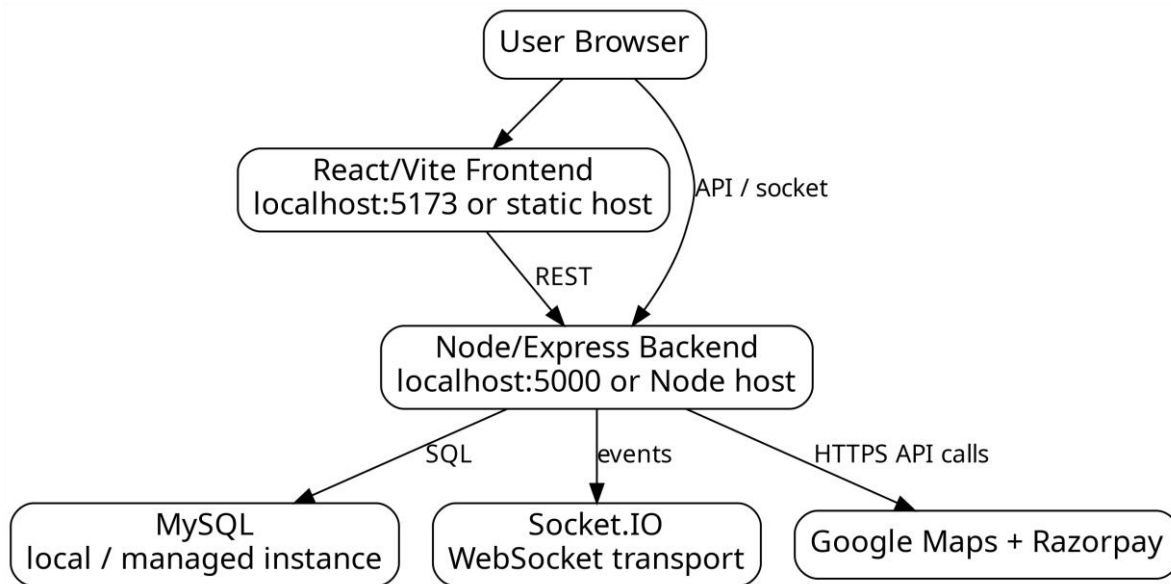


Figure 4.3 Local-to-cloud deployment architecture

4.9 Security and Failure Handling

4.9 Security and Failure Handling is considered an important part of the FoodFlow project because security and reliability depend on server-side validation, protected credentials, controlled state transitions, and clear error responses. The design is presented from both a user perspective and a database-systems perspective so that the

report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow should reject malformed input, unauthorized roles, invalid IDs, impossible quantities, and inconsistent order transitions before changing the database. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, when an external service fails, the platform should preserve database state and display a recoverable message rather than silently marking an action complete. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Hash passwords with bcryptjs.
- Keep API keys in environment variables.
- Validate totals and ownership on the server.
- Return meaningful HTTP status codes.

CHAPTER 5 – DESIGN

The chapter 5 – design subsection explains the design stage translates requirements into modules, entities, state machines, screens, and API contracts. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow is divided into functional modules whose responsibilities can be implemented and tested independently. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is design artefacts are included so that the report documents not only the final UI but also the reasoning behind database and interaction choices. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

5.1 System Design and Module Decomposition

The 5.1 system design and module decomposition discussion focuses on modular design limits the amount of code that must change when a single feature is modified. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the system is decomposed into Authentication, Restaurant Discovery, Menu Management, Cart/Checkout, Orders, Restaurant Operations, Delivery, Group Ordering, Split Bill, and Administration. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, modules communicate through explicit API functions and shared identifiers rather than direct cross-module database manipulation. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

FOODFLOW – ONLINE FOOD DELIVERY PLATFORM

Module	Primary Responsibility
Authentication	Login, tokens, role checks
Restaurant Discovery	Restaurant list and menu entry
Cart/Checkout	Cart state and order submission
Orders	Order details and history
Restaurant	Menu + preparation status
Delivery	Assignments + fulfilment
Group Order	Shared cart + participants
Admin	Onboarding + monitoring

5.2 Relational Schema Overview

5.2 Relational Schema Overview is considered an important part of the FoodFlow project because the relational schema is centered on stable identities and business transactions. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow uses normalized tables so a menu item is stored once and referenced by orders, while user and restaurant profiles are reused across operations. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, group-order tables extend the normal order model rather than duplicating user or menu definitions. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

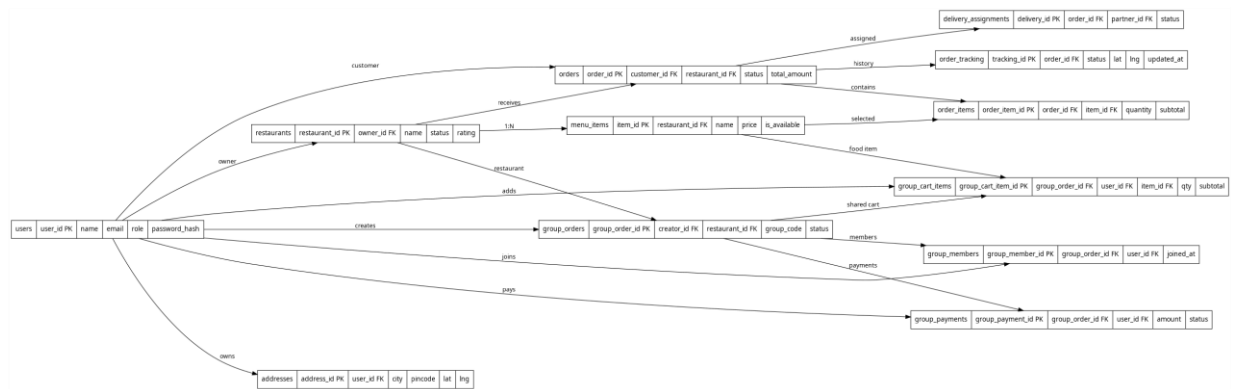


Figure 5.1 FoodFlow ER diagram / relational schema

5.2 Entity Details – Users and Restaurants

The 5.2 entity details – users and restaurants subsection explains user identity and restaurant ownership must be represented independently because a restaurant record contains business data while the user record contains authentication data. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the schema links restaurant ownership through an `owner_id` foreign key where required, while role information is stored in the user account. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is this separation lets the platform support different roles without copying login details into every business table. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

Entity	Key Fields	Design Purpose
users	<code>user_id</code> , <code>email</code> , <code>role</code> , <code>password_hash</code>	Authentication identity
restaurants	<code>restaurant_id</code> , <code>owner_id</code> , <code>name</code> , <code>status</code>	Restaurant business profile
menu_items	<code>item_id</code> , <code>restaurant_id</code> , <code>price</code> , <code>availability</code>	Food catalogue

5.2 Entity Details – Orders and Delivery

The 5.2 entity details – orders and delivery discussion focuses on orders represent the transaction header while `order_items` represent the many individual food selections. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, `delivery_assignments` connect an order to the delivery partner without embedding partner details repeatedly in the order row. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, `order_tracking` can preserve a history or latest status view so the customer sees a meaningful lifecycle. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

Table	Relationship
orders	Customer + restaurant + address
order_items	Order + menu item
delivery_assignments	Order + delivery partner
order_tracking	Order + status/location history

5.3 Group Order Data Model

5.3 Group Order Data Model is considered an important part of the FoodFlow project because group orders require membership, shared-cart attribution, and per-member payment state. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow stores `group_orders` as the parent record, `group_members` as participants, `group_cart_items` as member-owned food contributions, and `group_payments` as payment obligations. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, using separate child tables keeps the shared cart collaborative while preserving the identity of the user responsible for each item. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Table	Why It Exists
<code>group_orders</code>	Parent group context and restaurant
<code>group_members</code>	Participants in the group
<code>group_cart_items</code>	Shared cart with contributor identity
<code>group_payments</code>	Member-level amount and payment state

5.4 Group Split-Bill Logic

The 5.4 group split-bill logic subsection explains the split bill is calculated from item ownership rather than by blindly dividing the final amount by the number of members. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, for each participant, subtotal equals the sum of quantity multiplied by unit_price for that participant's cart rows; the group total is the sum of participant subtotals. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the design then creates or updates a payment record per member so the frontend can distinguish PAID and PENDING contributions. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

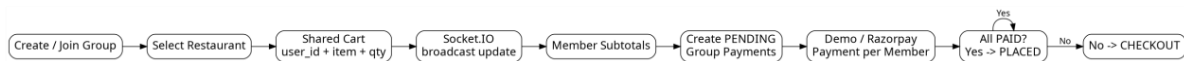


Figure 5.2 Split-bill processing logic

5.5 Order Status Design

The 5.5 order status design discussion focuses on order processing becomes predictable when valid states and state-changing roles are defined. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow uses the sequence ORDER_PLACED, ORDER_CONFIRMED, PREPARING, READY_FOR_PICKUP, OUT_FOR_DELIVERY, and DELIVERED as the documented lifecycle. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, restaurant actions govern preparation-related stages and delivery actions govern fulfilment stages, reducing accidental cross-role changes. The

approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

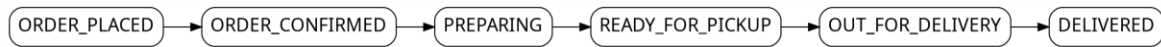


Figure 5.3 Documented order status lifecycle

5.6 UI Navigation Design

5.6 UI Navigation Design is considered an important part of the FoodFlow project because a food delivery platform must make the next important action obvious to each user. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, customer navigation centers on Home, Restaurants, Group Order, Track Order, Orders, Cart, and authentication actions, while restaurant and delivery users receive dedicated dashboards. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the Group Order interface uses Shared Cart, Add Food, and Split Bill views so collaboration can be understood as a sequence. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Consistent navigation labels.
- Role-specific dashboard entry points.
- Action buttons located near the related information.

5.7 API Contract and Validation Design

The 5.7 api contract and validation design subsection explains API contracts define what data is sent, what is returned, and which operations are allowed. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow endpoints should validate IDs, quantities, prices, ownership, status values, and required fields before issuing SQL mutations. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is consistent validation rules make frontend errors easier to diagnose and protect the database from malformed application input. The team

can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

Validation Area	Example Rule
Quantity	Must be a positive integer
Price	Server uses stored menu price
Ownership	Restaurant may edit only own records
Status	Only allowed next states are accepted
Role	Protected endpoints require matching role

CHAPTER 6 – IMPLEMENTATION

The chapter 6 – implementation discussion focuses on implementation converts the design into folders, components, API routes, SQL operations, and user workflows. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow separates frontend and backend code so each side can be installed and started independently. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the documentation describes implementation patterns that correspond to the project architecture and can be aligned with the exact source files during final submission. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

6.1 Project Structure and Local Setup

6.1 Project Structure and Local Setup is considered an important part of the FoodFlow project because a clear project directory makes it easier to find frontend pages, backend routes, database scripts, and environment files. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the current student workflow uses a Windows project directory with a frontend folder created through Vite and a backend folder for Node/Express services. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the root-level README should document the startup order so that a reviewer can reproduce the environment. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Folder / File Group	Purpose
frontend/	React + Vite application
backend/	Node.js + Express server
sql/ or database scripts	Schema and seed data
README.md	Setup and project explanation
.env	Local secrets and configuration
.gitignore	Prevent secrets and generated files from Git

6.2 Customer Module – Restaurant Discovery

The 6.2 customer module – restaurant discovery subsection explains customers start by browsing available restaurants and selecting a restaurant to view its menu. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the frontend requests restaurant data from the API and renders cards containing restaurant name, description, rating, and delivery information used in the demo interface. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the restaurant identifier then becomes the context for loading menu items and adding valid items to the cart. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

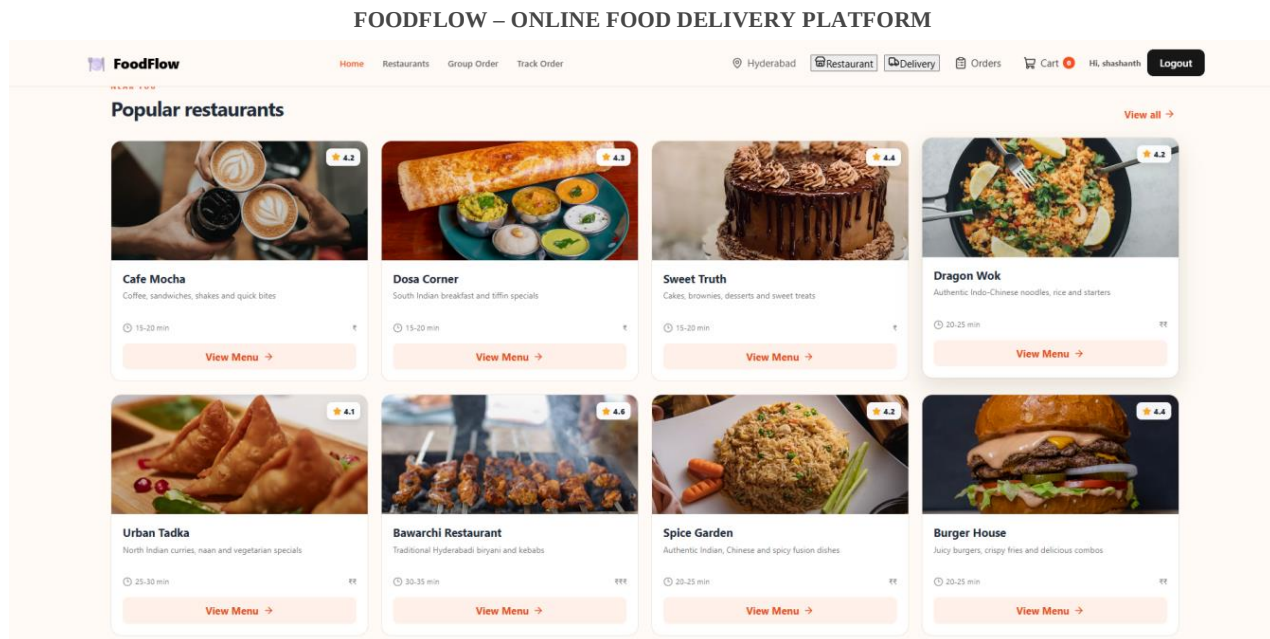


Figure 6.1 Customer restaurant discovery screen from the FoodFlow application

6.2 Customer Module – Menu and Cart

The 6.2 customer module – menu and cart discussion focuses on menu selection must preserve item identity, quantity, price, and restaurant context. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow displays menu items and allows the user to add items to a cart, update quantity, and review the calculated total before checkout. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the server should re-read the authoritative menu price when an order is submitted so client-side values are not trusted as the final charge. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Add item to cart.
- Increase or decrease quantity.
- Remove item.
- Validate availability before checkout.

6.2 Customer Module – Checkout and Order Placement

6.2 Customer Module – Checkout and Order Placement is considered an important part of the FoodFlow project because checkout combines the cart with a delivery address and payment choice to create a durable order. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the frontend sends order information to the backend, which validates the cart, computes the amount from stored data, inserts the order and line items, and returns the created order identifier. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the customer can then navigate to Order Details where the stored status and payment state are displayed. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Stage	Result
Review	Cart contents and amount
Address	Selected delivery location
Create	Order + order_items rows
Payment	Payment record/state
Track	Order Details page

6.3 Restaurant Module – Menu Management

The 6.3 restaurant module – menu management subsection explains restaurant users require CRUD operations for menu items so the displayed catalogue can remain current. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the FoodFlow dashboard can support create, read, update, and delete or availability-change operations tied to the restaurant identifier. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is server-side ownership checks are essential because the UI alone cannot prevent a user from attempting to modify another restaurant's data. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Create food item.
- Edit name, description, price, image, availability.
- Disable unavailable items.
- Review menu list.

6.3 Restaurant Module – Order Processing

The 6.3 restaurant module – order processing discussion focuses on restaurant operations turn a newly created order into an order that is ready for delivery. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the restaurant dashboard retrieves incoming orders and updates the preparation stages according to the documented lifecycle. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, after the restaurant marks the order ready for pickup, the delivery workflow can safely take over without bypassing the preparation state. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Review newly placed orders.
- Confirm / accept order.
- Mark as preparing.
- Mark ready for pickup.

6.4 Delivery Partner Module – Availability and Assignment

6.4 Delivery Partner Module – Availability and Assignment is considered an important part of the FoodFlow project because delivery partners need a list of work they are eligible to receive and a clear active-delivery view. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow represents the relationship through delivery assignment data and partner availability status. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the assignment record avoids putting partner-specific fields directly into customer data and supports a clear audit trail for which partner handled an order. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven

updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Set availability.
- View eligible assignments.
- Accept assignment.
- Open active delivery details.

6.4 Delivery Partner Module – Pickup, Route and Delivery

The 6.4 delivery partner module – pickup, route and delivery subsection explains delivery completion is a sequence rather than a single button. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the partner moves an order from ready for pickup to out for delivery and finally to delivered; location updates can be attached to the active delivery. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the frontend can display these changes to a customer through Order Details and Socket.IO events. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

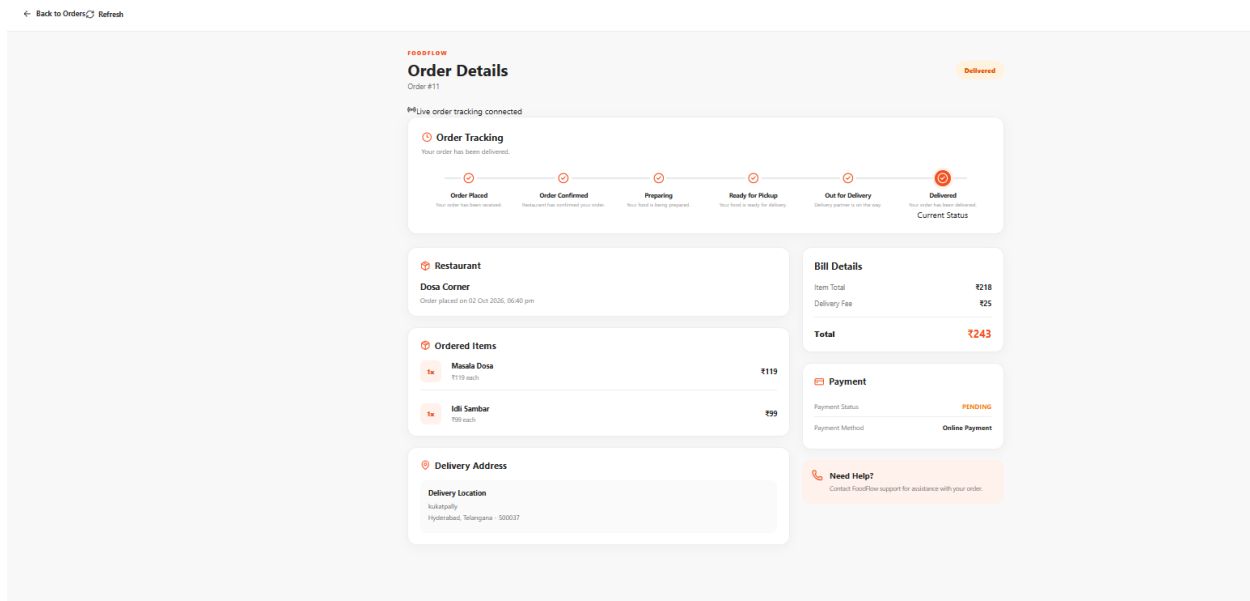


Figure 6.2 Order Details and delivery lifecycle screen

6.5 Admin Module – Onboarding and Commission Tracking

The 6.5 admin module – onboarding and commission tracking discussion focuses on administration requires cross-restaurant access and therefore stronger authorization than ordinary roles. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the planned FoodFlow admin layer supports restaurant onboarding, commission information, and platform-wide order monitoring. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, commission records can be stored per order or summarized per restaurant so the administrator can inspect operational activity without editing customer records directly. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

Admin Area	Representative Data
Onboarding	Restaurant name, owner, status
Commission	Order, rate, commission amount

FOODFLOW – ONLINE FOOD DELIVERY PLATFORM

Admin Area	Representative Data
Monitoring	Order ID, restaurant, status, amount
Operations	Platform-level filters and summaries

6.6 Group Order Module – Create and Join

6.6 Group Order Module – Create and Join is considered an important part of the FoodFlow project because a group order starts by creating a shared session with a unique group identifier or joining code. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow associates the group with a restaurant and records participants through `group_members`. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the UI should show the group identity clearly so multiple users can join the same session without ambiguity. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Create group.
- Select restaurant.
- Generate/share group code.
- Join group as participant.

6.7 Split Bill Implementation

The 6.7 split bill implementation subsection explains the split bill begins after the shared cart reaches a payable state. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow groups shared-cart records by `user_id`, calculates each member's subtotal, creates payment rows, and marks each row paid after the corresponding payment action is completed. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the group status changes to PLACED only when the required member payment rows are all paid. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

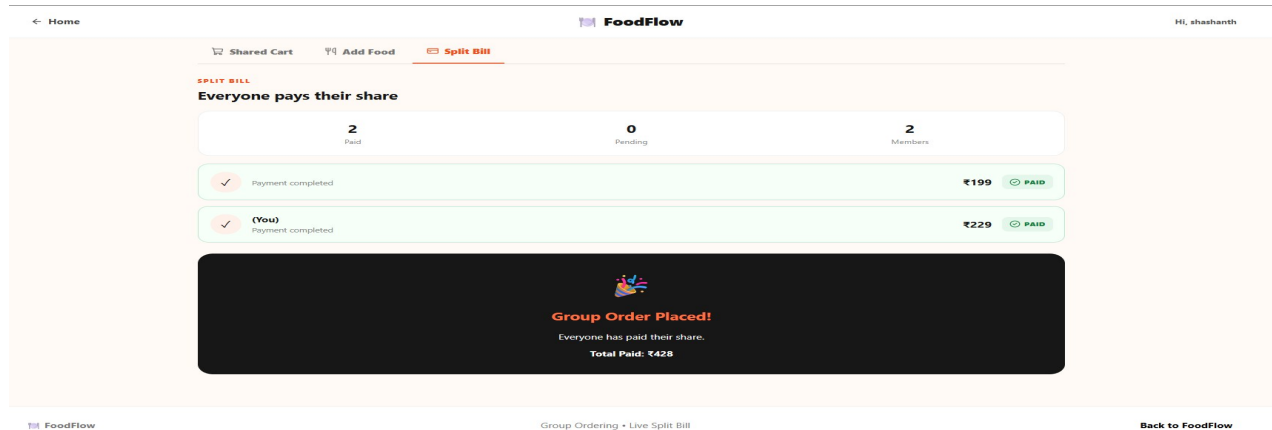


Figure 6.3 FoodFlow Split Bill result screen

6.8 API Service Layer

The 6.8 api service layer discussion focuses on centralizing HTTP functions prevents page components from repeating URLs, headers, and parsing logic. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow can expose service functions such as `getRestaurants`, `getRestaurantMenu`, `createOrder`, `getOrderDetails`, group-order actions, and delivery actions. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the service layer also provides one place to handle base URLs, error translation, authentication headers, and future API version changes. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

Service Function	Use
<code>getRestaurants()</code>	Restaurant discovery
<code>getRestaurantMenu(id)</code>	Menu retrieval
<code>createOrder(data)</code>	Order submission
<code>getOrderDetails(id)</code>	Tracking/details
<code>createGroupOrder(data)</code>	Group creation
<code>addGroupItem(id,data)</code>	Shared cart
<code>getSplitBill(id)</code>	Member payment state

6.9 Real-Time Event Integration and Error Handling

6.9 Real-Time Event Integration and Error Handling is considered an important part of the FoodFlow project because real-time UI updates require controlled subscriptions and clean-up when a component is unmounted. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow can listen for order-status, group-cart, group-payment, and delivery-location events while retaining periodic refresh or explicit fetch as a

synchronization fallback. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, error handling should surface network errors, invalid actions, and server-side validation failures without leaving stale optimistic state in the UI. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Connect once per relevant page/session.
- Remove listeners on cleanup.
- Refresh authoritative state after critical errors.
- Show user-friendly messages for rejected operations.

CHAPTER 7 – FEATURES

The chapter 7 – features subsection explains FoodFlow features are organized around real user tasks rather than isolated technical components. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the combined feature set demonstrates a complete food delivery lifecycle from restaurant discovery to delivery completion and group payment. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is each feature page documents both the visible behaviour and the underlying implementation responsibility. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

7.1 Role-Based Authentication

The 7.1 role-based authentication discussion focuses on authentication provides the entry point into protected workflows. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow separates customer, restaurant, delivery, and administrator contexts and associates each signed-in session with a role. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, role-based authorization reduces accidental access to unrelated functions and provides a clear review path during a demonstration. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Customer login and session.
- Restaurant login and dashboard.

- Delivery login and dashboard.
- Admin login and management context.

7.2 Restaurant Discovery and Search

7.2 Restaurant Discovery and Search is considered an important part of the FoodFlow project because customers need a quick way to compare restaurants before selecting one. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the FoodFlow home/restaurant interface uses cards with restaurant name, category/description, rating, delivery estimate, and a View Menu action. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the data is retrieved through the API so the same restaurant record can also appear in administrative and operational views. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Restaurant cards.
- Menu navigation.
- Search/filter-ready structure.
- Responsive layout.

7.3 Cart and Checkout

The 7.3 cart and checkout subsection explains the cart collects the exact items a customer intends to order before submission. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow supports quantity changes, total calculation, address selection, and payment initiation while keeping the restaurant context visible. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and

connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is checkout is treated as a controlled transaction boundary because it creates the order and associated child records. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Quantity control.
- Subtotal + delivery fee + total.
- Address selection.
- Payment state.

7.4 Order Tracking

The 7.4 order tracking discussion focuses on customers expect to know where an order is in the delivery lifecycle. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow uses an Order Details view with a visual status timeline and supporting cards for restaurant, ordered items, bill details, payment, and delivery address. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, Socket.IO events can update the timeline when the backend changes order state, while persistent data remains available through the order API. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

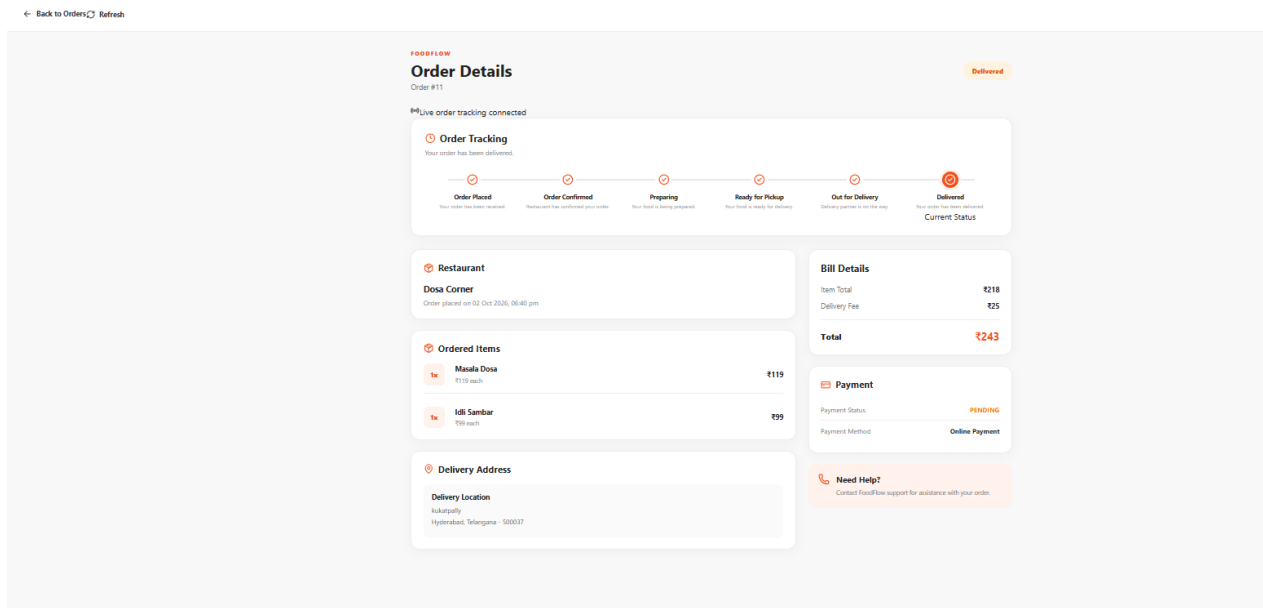


Figure 7.1 FoodFlow order tracking and details screen

7.5 Restaurant Order Management

7.5 Restaurant Order Management is considered an important part of the FoodFlow project because restaurant staff need operational visibility into new and active orders. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the restaurant dashboard separates preparation actions from customer actions and exposes the order states relevant to the kitchen workflow. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, server-side role and ownership checks ensure that status changes are made only by the responsible restaurant user. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Incoming order queue.
- Preparation status controls.
- Ready-for-pickup transition.
- Order history.

7.6 Delivery Operations

The 7.6 delivery operations subsection explains delivery partners need concise information about what to pick up, where to go, and which order is active. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow supports availability, assignment, pickup, out-for-delivery, delivered, and optional location-sharing states. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the delivery module therefore acts as the bridge between restaurant readiness and customer completion. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Available deliveries.
- Accepted / active deliveries.
- Pickup status.
- Delivered status.
- Location update path.

7.7 Group Ordering with Live Shared Cart

The 7.7 group ordering with live shared cart discussion focuses on group ordering allows multiple users to contribute to a single restaurant order. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow uses participant-aware cart rows so each member's contribution can be recalculated independently while Socket.IO keeps the shared view synchronized. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the interface separates Shared Cart, Add Food, and Split Bill so participants can understand the collaboration sequence. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

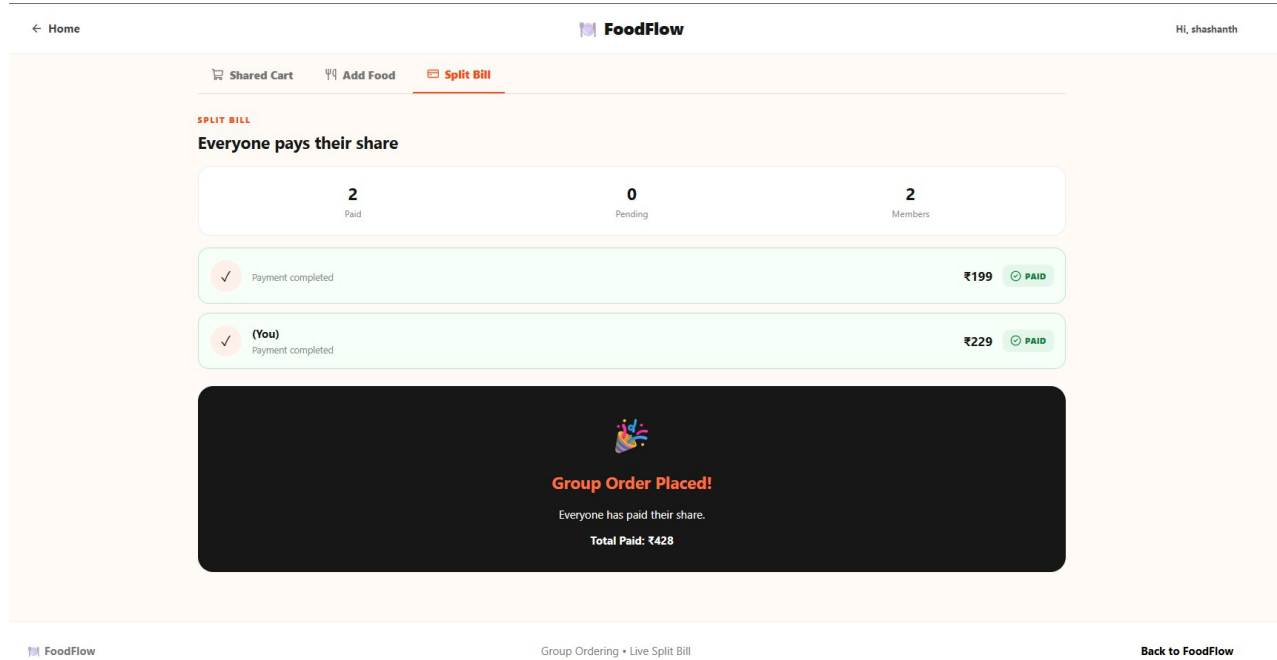


Figure 7.2 Live Split Bill view used for the differentiating feature

7.8 Split Bill and Administrative Monitoring

7.8 Split Bill and Administrative Monitoring is considered an important part of the FoodFlow project because split billing turns a single group total into participant-level payment states while administration provides platform-level oversight. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow records each member contribution and payment status separately; the admin layer can use the same normalized data to monitor order and commission information. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, this feature pair demonstrates that the same relational data can support both user-level interaction and management-level reporting. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Feature	User View	Technical View
Split bill	Each member sees own share	group_cart_items + group_payments
Commission	Admin sees platform amount	order-linked commission records
Monitoring	Admin sees order status	cross-table JOIN queries
Completion	Group placed after all paid	controlled status transition

CHAPTER 8 – TESTING

The chapter 8 – testing subsection explains testing verifies that the separate modules work correctly and also cooperate as one workflow. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow testing covers API requests, UI navigation, database persistence, real-time events, group split calculations, and role restrictions. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the report records expected results and includes a set of repeatable test cases for academic verification. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

8.1 Testing Strategy

The 8.1 testing strategy discussion focuses on testing should begin with individual requests and grow into end-to-end workflows. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow can be tested at four levels: API behaviour, module behaviour, integration behaviour, and user-interface flow. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, using different test levels helps isolate defects because a failing screen can be traced to either UI state, API response, SQL data, or event handling. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

Level	Example
API	Login, restaurant list, order creation
Module	Cart totals, menu CRUD
Integration	Restaurant -> delivery -> customer
Real-time	Order status and group cart events
UI	Navigation and button behaviour

8.2 API / Postman Testing

8.2 API / Postman Testing is considered an important part of the FoodFlow project because Postman provides a direct way to verify backend behaviour without relying on the frontend. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow endpoints such as login, restaurant listing, menu retrieval, order creation, status updates, and group operations can be sent with controlled input data. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the test workspace should keep request examples organized by role and should verify status codes as well as response bodies. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Verify success and validation responses.
- Check required headers and authentication.
- Confirm JSON response structure.
- Repeat important requests after database reset.

8.3 Integration Testing

The 8.3 integration testing subsection explains integration testing checks whether one module hands correct information to another. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and

operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, a typical FoodFlow integration scenario moves from customer checkout to restaurant processing, delivery pickup, customer tracking, and final delivery. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is group ordering is a second integration path in which several clients share one state, a payment is recorded per member, and completion changes the group state. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Customer -> Restaurant.
- Restaurant -> Delivery.
- Delivery -> Customer tracking.
- Group member -> Shared cart -> Split payment.

8.4 Validation and Negative Testing

The 8.4 validation and negative testing discussion focuses on negative tests deliberately send invalid data to confirm that the backend rejects unsafe or impossible operations. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow should reject missing fields, zero/negative quantities, invalid restaurant or user IDs, unauthorized roles, and illegal order status transitions. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, negative testing is especially important for payment and group-order states because a false success could produce inconsistent business records. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

Scenario	Expected Behaviour
Invalid login	401/appropriate error
Negative quantity	Validation error; no insert
Wrong role	403/authorization error
Unknown order	404/not found
Illegal status	400/validation error

8.5 Test Environment and Data Preparation

8.5 Test Environment and Data Preparation is considered an important part of the FoodFlow project because repeatable testing requires a known environment and predictable records. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the academic environment uses Windows, VS Code, Node.js, MySQL, a modern browser, and optionally Postman. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, test data should include at least one customer, one restaurant, several menu items, one delivery partner, and a two-member group order so that all main flows can be demonstrated. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Data Set	Purpose
Customer account	Normal order
Restaurant account	Menu/order processing
Delivery account	Fulfilment
Menu items	Cart and group cart
Group members	Split-bill test

Data Set	Purpose
Order records	Tracking / history

8.6 Test Cases – Authentication and Restaurant Data

The 8.6 test cases – authentication and restaurant data subsection explains authentication and discovery are the first verifiable stages of the workflow. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, these tests confirm that the right user can sign in, receive a valid session, retrieve restaurant data, and access a menu. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is successful results should be checked both visually and through database/API responses where possible. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

TC	Test	Expected Result	Result
TC01	Valid customer login	Dashboard opens	Pass
TC02	Invalid password	Login rejected	Pass
TC03	Restaurant list	Restaurant cards load	Pass
TC04	Menu retrieval	Selected menu loads	Pass
TC05	Unauthorized route	Access denied / redirected	Pass

8.6 Test Cases – Cart, Checkout and Orders

The 8.6 test cases – cart, checkout and orders discussion focuses on cart and checkout tests verify the transition from temporary UI state to durable order state. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the system should calculate the displayed total correctly, create one order header with the expected line items, and preserve the selected delivery address. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, after creation, the Order Details view should show the same restaurant, items, amount, and initial order status. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

TC	Test	Expected Result	Result
TC06	Add item	Item appears in cart	Pass
TC07	Change quantity	Subtotal recalculates	Pass
TC08	Create order	Order + items inserted	Pass
TC09	Open order details	Stored details displayed	Pass
TC10	Invalid item ID	Request rejected	Pass

8.6 Test Cases – Restaurant and Delivery Workflow

8.6 Test Cases – Restaurant and Delivery Workflow is considered an important part of the FoodFlow project because this set verifies the operational side of the order lifecycle. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the restaurant should be able to move an eligible order through confirmation, preparation, and ready-for-pickup states while the delivery partner receives only the work it can legitimately accept. The implementation separates

presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, database rows and UI state should remain aligned after every transition. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

TC	Test	Expected Result	Result
TC11	Restaurant views order	Order appears	Pass
TC12	Mark preparing	Status updated	Pass
TC13	Mark ready	Ready for pickup	Pass
TC14	Delivery accepts	Assignment active	Pass
TC15	Deliver order	Final status delivered	Pass

8.6 Test Cases – Group Order and Split Bill

The 8.6 test cases – group order and split bill subsection explains group-order testing must verify synchronization as well as arithmetic. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, participants should see the same shared cart after an item is added, each member should receive the correct subtotal, and payment rows should change independently. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the final group state should become placed only after the required participant payment conditions are met. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event

observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

TC	Test	Expected Result	Result
TC16	Create group	Group ID/code created	Pass
TC17	Join group	Member visible to group	Pass
TC18	Add shared item	Cart updates for members	Pass
TC19	Calculate shares	Subtotals match items	Pass
TC20	All member payments	Group becomes placed	Pass

8.7 Test Results and Observations

The 8.7 test results and observations discussion focuses on the documented test set covers the major user journeys and the differentiating group feature. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the results column is intended to be updated with the final execution evidence during the demonstration cycle; the current report records the expected academic verification set and the demonstrated behaviours documented by the team. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, a final test run should also include browser responsiveness, API error paths, database reset/reseed, and external service configuration checks. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Role-based login flow verified.
- Restaurant/menu/order CRUD path verified.
- Delivery lifecycle verified.

- Group split totals and member payment state verified.
- External integrations should be retested when credentials are configured.

CHAPTER 9 – DEPLOYMENT

CHAPTER 9 – DEPLOYMENT is considered an important part of the FoodFlow project because deployment moves the locally working components into a reproducible runtime arrangement. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the current academic environment is local, while the architecture supports a future split deployment of frontend, backend, and MySQL. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, deployment instructions therefore distinguish local startup from production hosting and external integration setup. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

9.1 Deployment Steps – Local Environment

The 9.1 deployment steps – local environment subsection explains the simplest academic deployment runs frontend, backend, and MySQL on the same development machine. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow should be installed with npm dependencies, database schema/seed data, environment variables, backend startup, and frontend startup in that order. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the browser then opens the Vite-served frontend URL and communicates with the local Express server. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system

easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Install Node.js and npm.
- Install MySQL.
- Install frontend/backend dependencies.
- Create database and run schema.
- Configure .env files.
- Start backend, then frontend.

9.2 Environment Configuration

The 9.2 environment configuration discussion focuses on environment variables prevent hard-coded secrets and deployment addresses from being scattered through source code. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, FoodFlow needs configuration for MySQL connection data, JWT/session secrets, backend and frontend base URLs, Socket.IO origin rules, and optional Maps/Razorpay credentials. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, development and production should use different secrets and should never commit .env files to the repository. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

Variable Type	Example Purpose
DB_*	MySQL connection
JWT_SECRET	Token signing
FRONTEND_URL	CORS / allowed origin
VITE_API_BASE_URL	Frontend backend base URL
MAPS_API_KEY	Map integration
RAZORPAY_*	Payment integration

9.3 Local Run Procedure in VS Code

9.3 Local Run Procedure in VS Code is considered an important part of the FoodFlow project because the project is organized so frontend and backend can be started from separate terminals. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the student workflow uses the frontend directory for the Vite command and the backend directory for the Node/Express command, while MySQL runs as an independent service. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, this separation also makes debugging clearer because UI errors, API errors, and database errors appear in different logs. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Component	Typical Local Address	Role
Frontend	http://localhost:5173	React UI
Backend	http://localhost:5000	REST + Socket.IO
Health API	http://localhost:5000/api/health	Backend check
MySQL	localhost:3306	Data persistence

9.4 Front-End Hosting Considerations

The 9.4 front-end hosting considerations subsection explains React/Vite can be built into static assets that a web host can serve. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, deployment must update the API base URL so the browser points to the hosted backend and the allowed origins must include the deployed frontend domain.

The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is client-side routing also needs a fallback configuration on hosts that serve a single-page application. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Run production build.
- Set the API URL in environment configuration.
- Enable SPA route fallback.
- Use HTTPS for the hosted application.

9.5 Back-End Hosting and Production Checklist

The 9.5 back-end hosting and production checklist discussion focuses on a Node/Express backend requires a persistent runtime or compatible serverless arrangement and secure environment configuration. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the backend should use a managed MySQL database or a protected database server, set CORS correctly, keep secrets outside source control, and use HTTPS. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, external payment and map credentials should be restricted and rotated according to the deployment environment. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Use HTTPS and secure headers.
- Restrict CORS origins.
- Use managed secrets.
- Backup database regularly.

- Monitor server logs and failures.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

CHAPTER 10 – CHALLENGES AND LIMITATIONS is considered an important part of the FoodFlow project because the main engineering challenges come from integrating multiple roles and stateful workflows rather than from a single isolated screen. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow requires consistency across customer, restaurant, delivery, and group-order views. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, documenting the challenges is useful because it shows how design decisions were made and what remains to be improved. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

10.1 Challenge – Keeping Order States Consistent

The 10.1 challenge – keeping order states consistent subsection explains an order can be viewed by several roles and each role is responsible for different transitions. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow addresses this by defining a controlled lifecycle and restricting transitions according to role and current status. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is this approach prevents cases such as a delivery partner collecting an order that has not been marked ready or a completed order moving backward unexpectedly. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered

verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Define allowed states.
- Validate current state before update.
- Emit real-time event after database update.

10.1 Challenge – Synchronizing Group Cart Changes

The 10.1 challenge – synchronizing group cart changes discussion focuses on a shared cart can be edited by several participants at the same time. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, Socket.IO provides low-latency event communication, while the database stores the authoritative cart rows and the frontend can perform periodic refresh as a fallback. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the design therefore treats real-time updates as notifications that lead back to an authoritative state rather than as the sole persistence mechanism. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Broadcast item changes.
- Preserve contributor user_id.
- Refresh if an event is missed.

10.2 Challenge – External Services and Payment Safety

10.2 Challenge – External Services and Payment Safety is considered an important part of the FoodFlow project because Google Maps and Razorpay depend on external credentials, network access, provider configuration, and production security practices. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the academic report separates the external integration interfaces from the local demonstration payment flow so evaluation does not require actual money

movement. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, this reduces deployment risk during review while keeping the architecture ready for credentialed integration later. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Use test/sandbox credentials when available.
- Never hard-code secrets.
- Treat external failure as recoverable.

10.3 Current Limitations and Known Issues

The 10.3 current limitations and known issues subsection explains the current academic prototype is primarily intended for local demonstration and therefore has boundaries compared with a production platform. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, live payment settlement, cloud deployment, complete map routing, large-scale concurrency testing, notification services, and enterprise-grade monitoring are future concerns. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is these are documented limitations rather than hidden assumptions, allowing the evaluation to focus on what the current project actually demonstrates. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

FOODFLOW – ONLINE FOOD DELIVERY PLATFORM

Area	Current Academic Boundary
Payments	Demo flow unless valid Razorpay integration is configured
Maps	Integration point; full live routing requires credentials
Scale	Local development dataset / semester scope
Notifications	Web UI events; production push/SMS not included
Security	Academic implementation; production hardening remains required

CHAPTER 11 – FUTURE ENHANCEMENTS

The chapter 11 – future enhancements discussion focuses on the architecture intentionally leaves extension points for production-oriented capabilities. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, future work should build on the existing relational model and API boundaries instead of duplicating the system. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the proposed enhancements focus on payments, mapping, analytics, performance, security, and intelligent assistance. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

11.1 Planned Features – Live Maps and Payment Gateway

11.1 Planned Features – Live Maps and Payment Gateway is considered an important part of the FoodFlow project because map-based tracking and real payment integration are the most visible production extensions. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, FoodFlow can connect the delivery location fields to Google Maps rendering and use Razorpay order/payment APIs with secure server-side verification. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the group split-bill model can remain participant-oriented while the gateway layer processes individual payment references. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Map with driver location marker.
- Route or ETA estimation.
- Razorpay order creation and signature verification.
- Payment reconciliation records.

11.2 Analytics and Administrative Intelligence

The 11.2 analytics and administrative intelligence subsection explains once platform data grows, administrators need summaries rather than individual records. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow can generate metrics such as daily orders, restaurant activity, average preparation time, delivery completion time, commission totals, and group-order adoption. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is analytics should be derived from normalized transactional data and stored summary tables only when performance requires them. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- Daily/weekly order dashboards.
- Restaurant performance metrics.
- Commission and revenue summaries.
- Group-order participation analytics.

11.3 Performance, Scalability and Security

The 11.3 performance, scalability and security discussion focuses on a cloud deployment must handle more simultaneous users, larger order histories, and more real-time connections than a local academic demo. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, future improvements include database indexing, connection pooling, caching for restaurant menus, queue-based background work, horizontal backend scaling, and stronger authentication controls. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, security can be strengthened with refresh-token strategies, input sanitization, rate limiting, audit logs, and centralized secret management. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Connection pooling and indexes.
- Caching frequently read menus.
- Rate limiting and audit logs.
- Scalable Socket.IO infrastructure.

11.4 Intelligent Enhancements and Sustainability

11.4 Intelligent Enhancements and Sustainability is considered an important part of the FoodFlow project because the platform can later use its accumulated data to support recommendation and operational optimization features. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, examples include personalized restaurant suggestions, demand-aware preparation estimates, delivery-partner workload balancing, and anomaly detection for failed or repeated payment events. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, such features should remain explainable and should not bypass the deterministic transactional rules that protect order and payment consistency. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Restaurant/item recommendations.

- Delivery workload balancing.
- ETA prediction.
- Anomaly and fraud signals.
- Sustainable route planning.

CHAPTER 12 – CONCLUSION

The chapter 12 – conclusion subsection explains the project integrates a complete food delivery workflow within one database-backed full-stack system. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow connects customers, restaurants, delivery partners, and administration through role-based interfaces and shared relational records. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the result is an academic demonstration of practical DBMS engineering combined with frontend, backend, and real-time communication. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

12.1 Summary of the Project

The 12.1 summary of the project discussion focuses on the report has documented requirements, architecture, design, implementation, features, testing, deployment, limitations, and future enhancements. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the core workflow supports restaurant discovery, menu selection, cart and checkout, order management, delivery fulfilment, tracking, and group split billing. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the project uses a consistent data and service model so the same order can be followed from creation to delivery. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- Integrated multi-role workflow.
- MySQL relational persistence.
- REST + Socket.IO backend communication.

12.2 Achievements and 12.3 Skills Learned

12.2 Achievements and 12.3 Skills Learned is considered an important part of the FoodFlow project because the project demonstrates the ability to convert a real-world requirement into a structured system with database entities, APIs, interfaces, and test cases. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the team gains practical experience in React component design, Node/Express services, SQL schema design, REST APIs, authentication, real-time events, and documentation. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the Group Ordering with Live Split Bill feature adds collaborative data handling and participant-level payment reasoning to the project. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Area	Skill Demonstrated
DBMS	Schema, keys, joins, normalization
Web	React, CSS, REST APIs
Backend	Express, validation, middleware
Real-time	Socket.IO event design
Testing	Postman, integration, negative tests
Engineering	Modular design, documentation, deployment

CHAPTER 13 – REFERENCES

The chapter 13 – references subsection explains a project report should make its technical foundations traceable to reliable learning and developer resources. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the references used for FoodFlow are primarily official documentation and standard learning material for the selected stack. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is official references are preferred for API syntax and framework behaviour, while textbooks and tutorials provide conceptual background. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

13.1 Books, Tutorials and Framework Documentation

The 13.1 books, tutorials and framework documentation discussion focuses on the following categories of sources support the conceptual and implementation parts of the project. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, React and Vite documentation inform component and development-tool usage; Node.js and Express documentation inform server-side JavaScript and routing; MySQL documentation informs relational queries and schema design. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, Socket.IO documentation supports event-based communication, while Postman resources support repeatable API testing. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- React documentation and guides.
- Vite documentation.
- Node.js documentation.
- Express.js documentation.
- MySQL 8.0 documentation.
- Socket.IO documentation.
- Postman learning resources.

13.2 APIs, External Services and Supporting Resources

13.2 APIs, External Services and Supporting Resources is considered an important part of the FoodFlow project because external service documentation is necessary when enabling maps or payment capabilities. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, Google Maps Platform documentation defines map and location-service usage, while Razorpay documentation defines gateway integration and server-side verification requirements. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, GitHub documentation and standard Git references support repository management, collaboration, branching, and secure handling of environment files. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Resource	Application in the Project
Google Maps Platform	Map and location integration
Razorpay	Payment gateway integration
Git/GitHub	Version control and collaboration
MDN Web Docs	Browser and web platform reference
Postman	REST API testing

CHAPTER 14 – APPENDICES

The chapter 14 – appendices subsection explains appendices contain the evidence and practical material that supports the main chapters without interrupting the conceptual narrative. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, FoodFlow appendices include application screenshots, representative code snippets, installation instructions, user guidance, academic review material, and feedback/signature pages. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is these artefacts make the report more useful as both a submission document and a reproduction guide. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

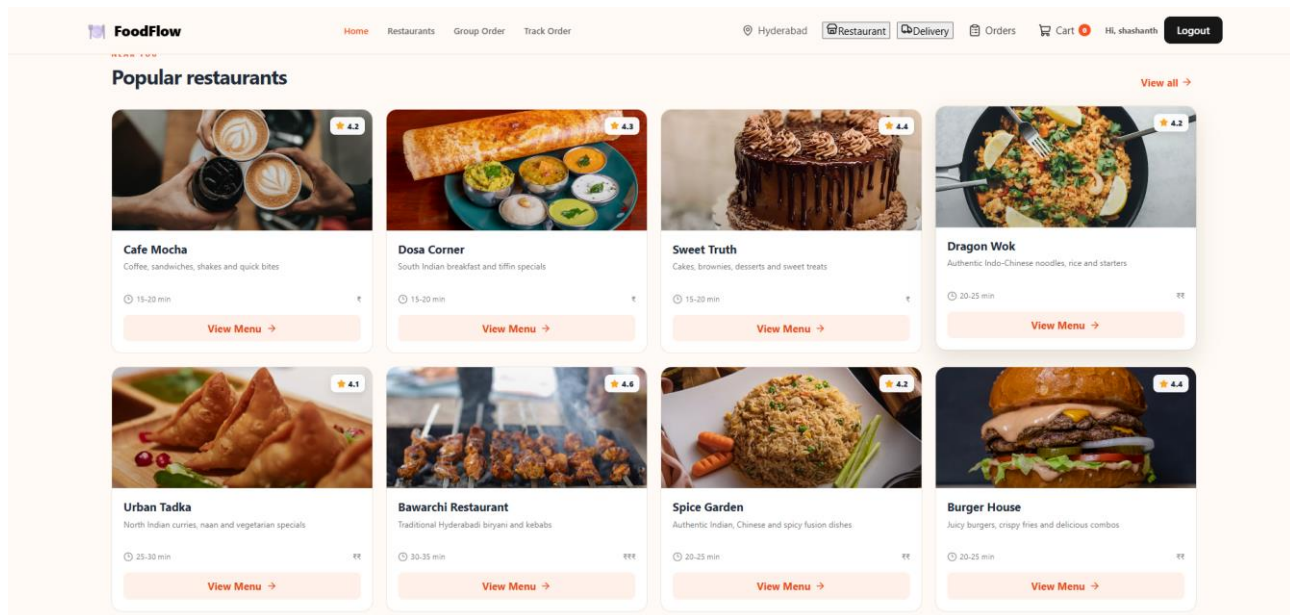
Appendix A – Application Screenshot: Restaurant Discovery

The appendix a – application screenshot: restaurant discovery discussion focuses on the restaurant discovery screen demonstrates the customer-facing visual layer of FoodFlow. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, the screenshot shows restaurant cards, ratings, delivery-time indicators, and navigation controls that connect users to menu pages. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the figure is included as implementation evidence rather than as a generic mock-up. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express

routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.



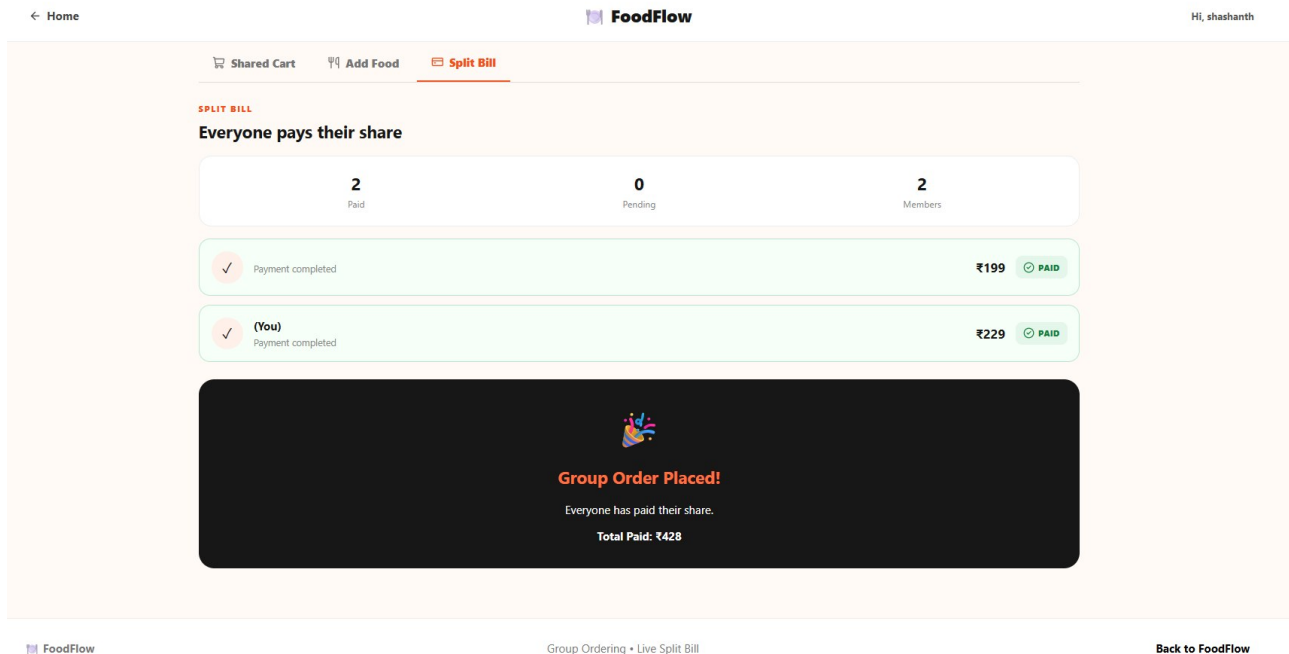
Appendix Figure A.1 Restaurant discovery screen

Appendix A – Application Screenshot: Group Split Bill

Appendix A – Application Screenshot: Group Split Bill is considered an important part of the FoodFlow project because the split-bill screen demonstrates the differentiating collaborative feature. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, the screenshot shows paid member rows, totals, member counts, and a final group-order state after all payments are completed. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, this evidence directly supports the Group Order and Split Bill sections of the report. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.



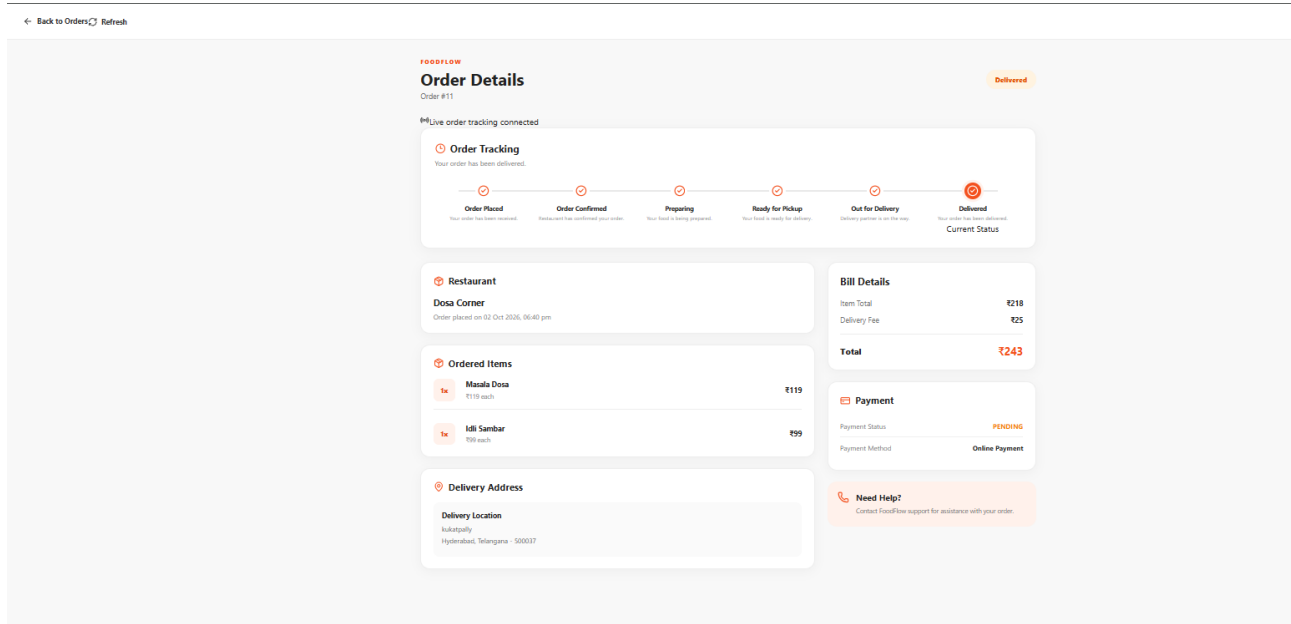
Appendix Figure A.2 Group Ordering with Live Split Bill screen

Appendix A – Application Screenshot: Order Details

The appendix a – application screenshot: order details subsection explains the Order Details page brings together order status, restaurant information, ordered items, bill details, payment state, and delivery address. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the screen demonstrates how persisted relational data is assembled into a single customer-facing view. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the status timeline also provides a clear visual explanation of the order lifecycle documented in Chapter 5. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.



Appendix Figure A.3 Order Details and tracking screen

Appendix B – Representative Code Snippet: Express API

The appendix b – representative code snippet: express api discussion focuses on a representative backend route illustrates how FoodFlow maps an HTTP request to a controller operation. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, a production implementation should validate authentication, ownership, input types, current state, and database errors before returning a success response. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the snippet is included as documentation of the design pattern and should be aligned with the final source code during project submission. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

- POST /api/orders receives customer checkout data.
- Controller validates cart and computes amount from server-side menu data.
- SQL inserts order header and order items.
- JSON response returns the new order identifier.

Appendix B – Representative Code Snippet: Socket.IO and Split Bill

Appendix B – Representative Code Snippet: Socket.IO and Split Bill is considered an important part of the FoodFlow project because the real-time layer can be expressed as a small set of event handlers and server emissions. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, when a group cart item is added, the backend stores the item and emits a `group:cart-updated` event to interested clients; when a member pays, `group-payment-updated` can notify the group view. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, the calculation remains database-backed so clients do not become the authoritative source of the payable amount. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

- Join a group-specific room.
- Persist cart item before broadcasting.
- Recompute member subtotal from stored rows.
- Emit payment state after persistence.

Appendix C – Installation / Setup Instructions

The appendix c – installation / setup instructions subsection explains the installation process should be repeatable on the student development machine. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the expected sequence is to open the project in VS Code, install frontend/backend dependencies, start MySQL, create the database, load schema/seed data, configure environment variables, and start both application servers. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is a health endpoint and browser URL provide quick checks before the full workflow is demonstrated. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

- 1. Open project in VS Code.
- 2. Verify node --version and npm --version.
- 3. Install dependencies in frontend and backend.
- 4. Create MySQL database and run schema.
- 5. Configure .env.
- 6. Start backend on port 5000.
- 7. Start frontend on port 5173.
- 8. Test role-based login and API health.

Appendix D – User Guide

The appendix d – user guide discussion focuses on a short user guide helps reviewers understand the sequence of actions without reading the implementation chapters first. This requirement influences both the user interface and the data model, because a visible action such as placing an order or joining a group must correspond to a controlled server-side operation and a durable database state.

In FoodFlow, each role has a distinct workflow, while group ordering introduces an additional collaborative sequence. The application records the information needed for later retrieval, reporting, status tracking, and audit-style examination. The frontend presents concise status information while the backend remains responsible for validating requests and applying consistent updates.

For academic evaluation, the guide can be used during the project demonstration as a checklist. The approach provides a direct mapping between the stated project objectives and demonstrable technical artefacts such as React pages, Express routes, MySQL tables, Socket.IO events, screenshots, test cases, and deployment instructions.

Role	Quick Workflow
Customer	Login -> Browse -> Menu -> Cart -> Checkout -> Track
Restaurant	Login -> Menu -> Incoming Orders -> Prepare -> Ready
Delivery	Login -> Available Orders -> Accept -> Pickup -> Deliver
Admin	Login -> Onboarding -> Commission -> Orders
Group Members	Create/Join -> Add Food -> Review Shares -> Pay

Appendix E – Academic Review / Demonstration Checklist

Appendix E – Academic Review / Demonstration Checklist is considered an important part of the FoodFlow project because the project review should demonstrate not only the UI but also the technical foundation behind it. The design is presented from both a user perspective and a database-systems perspective so that the report connects visible application behaviour with the underlying data and service responsibilities.

Within FoodFlow, reviewers can inspect MySQL schema and keys, Express routes, real-time events, role-based dashboards, selected test cases, and the Group Split Bill differentiator. The implementation separates presentation, business processing, persistence, and real-time communication so that changes in one area can be tested without rewriting the complete platform. This separation also supports the four role-oriented workflows used in the project: customer, restaurant, delivery partner, and administrator.

From a technical and academic viewpoint, external Maps and Razorpay features should be demonstrated only when the required configuration is available. The design therefore demonstrates practical use of relational modelling, RESTful communication, role-specific authorization, event-driven updates, validation, error handling, and transaction-aware state changes. These concepts are especially visible in the order lifecycle and the Group Ordering with Live Split Bill module.

Area	Demonstration Point
Database	Show schema, keys, sample order/group records
Backend	Show routes, validation, database queries
Real-Time	Show order or group update event
Maps	Show live map if configured
Payment	Show sandbox/test flow if configured
Roles	Customer, restaurant, delivery, admin screens
Differentiator	Group Ordering with Live Split Bill
Testing	Postman requests and test cases

Appendix F – Feedback, Signatures and Final Submission Notes

The appendix f – feedback, signatures and final submission notes subsection explains the final appendix provides space for academic review comments and formal signatures. In a full-stack application, this area is not isolated: it interacts with browser state, backend endpoints, database records, and operational rules. Documenting the interaction is necessary for reproducibility and for explaining the project during an academic review.

For FoodFlow, the report should be updated with the final tested screenshots, exact source-file references, and any deployment evidence available at the time of submission. The current academic build is intended for local demonstration on Windows, VS Code, Node.js, and MySQL, with external services configured only when valid credentials and connectivity are available. The documentation therefore distinguishes implemented local behaviour from integration points intended for production-style deployment.

The key engineering consideration is the team can use this page to record review feedback and planned corrections before final viva or project evaluation. The team can verify the expected behaviour using browser tests, API calls, MySQL queries, and real-time event observation. This layered verification reduces debugging time and makes the system easier to extend when additional restaurants, users, delivery operations, or payment scenarios are introduced.

Item	Entry
Guide Remarks	_____
Course Coordinator	_____
HOD / Department	_____
Student Signatures	1. _____ 2. _____ 3. _____
Final Review Date	_____

Submission note: Replace any placeholder configuration details with the exact final project values, attach the final source repository, and keep secrets out of the submitted codebase.

Installation/setup instructions The GOGlobe Integrated Tourism Platform is simple to install because it is built entirely using HTML, CSS, and JavaScript. It does not require any external frameworks or databases. The setup can be completed in just a few steps.

First, download or copy the complete GOglobe project folder to your local computer. If it is shared as a ZIPfile, extract it using any standard extraction tool. Make sure all files especially index.html, style.css, and script.js (if included) are in the same folder. Next, open the folder in a text editor like Visual Studio Code or Sublime Text to review or modify the code. To run the project, simply double-click on the index.html file or right-click and select “Open with Browser.” The website will open and function in any modern browser such as Google Chrome, Microsoft Edge, or Firefox. No installations or dependencies are needed. The project is fully self-contained and works offline, making it ideal for classroom demonstrations and educational submissions.

User manual or guide When you open the GOglobe platform, the homepage displays the main navigation bar with options like Login, Register, Destinations, and Travel Assistance. Users can begin by clicking the Login or Register buttons. A clean, half-page form will appear where users can enter their details to simulate login or registration. Once logged in, users can explore Top Global Destinations, view beautiful travel images, and get information about each place. The Travel Assistance section offers quick options like medical help, hotel booking, and transportation support all designed for educational simulation. Navigation through the platform is smooth, with buttons that automatically scroll to their respective sections. Alerts confirm actions such as successful login or registration. The entire experience demonstrates how an integrated tourism platform would look and behave in a real-world environment.